DECISION SERVICES FOR CODING AGENTS

htrnguyen
AIAI Laboratory

aisq.science htrnguyen.id.vn

ABSTRACT

Coding agents can generate plans and patches, but many surrounding choices have a small, explicit answer space. This report examines when those choices merit a separate decision service, when deterministic code is preferable, and how uncertainty should affect the workflow. It synthesizes selected research and engineering guidance, then introduces Laya MCP Server as an integration designed to make bounded decisions accessible through a common tool interface. It presents a task-selection framework, an integration contract and a reproducible workflow, illustrated with successful and incorrect predictions. It does not claim new model training, a general benchmark, equivalent quality to Jev or guaranteed productivity gains.

1 SCOPE AND METHOD

This is a targeted narrative survey, not a systematic literature review. Sources were selected for direct relevance to agent composition, model routing, abstention, context selection and evaluation. Primary research and official engineering guidance are separated from the Jev explainer and from observations of this project. Sources were consulted on 04 October 2026.

The intended reader is an engineer deciding how to incorporate a bounded judgment tool into an existing coding workflow. The discussion is deployment-neutral; installation details remain in the project README.

A decision service is useful when it improves a measurable part of the workflow. The agent still needs to verify the resulting change.

2 RELATED WORK

Theme / source	Finding relevant to integration
Workflow composition [7]	Anthropic distinguishes predefined workflows from model-directed agents and recommends beginning with simple compositions.
Bounded judgments [1]	The Jev explainer frames semantic decisions as an explicit interface alongside a generative model.
Learned routing [8]	RouteLLM trains routers using preference data to choose between stronger and weaker models. Its measured trade-offs are specific to its experiments.
Selective prediction [9]	SelectiveNet jointly learns prediction and rejection. It motivates measuring the quality of accepted cases as coverage changes.
Context selection [10]	Context engineering treats the information supplied to a model as a managed resource, not simply a longer prompt.
Agent evaluation [11]	Agent evaluations need task outcomes and execution evidence; a plausible response alone is insufficient.

Table 1: Related work and synthesis

These sources suggest defining the question and its interface before deciding which cases may proceed without additional review. Evaluation then needs to cover the complete workflow. These

sources justify investigating that design; they do not establish the accuracy of this Laya checkpoint.

This project uses neither the learned RouteLLM router nor the SelectiveNet architecture. We use those studies to guide evaluation of routing trade-offs and rejection behavior; their reported scores and thresholds do not carry over to Laya.

Schema validation checks response structure. Model evaluation measures decision correctness on relevant cases. Workflow evaluation measures the effect on task outcomes. Each requires its own evidence.

3 APPLICATIONS

Application	Bounded question	Evidence status here
Support routing	Which known issue class matches this message?	One explicit example observed.
Log triage	Which failure is directly supported by the excerpt?	One explicit and one ambiguous example observed.
Retrieval filtering	Does this passage address the supplied question?	Integration pattern; not task-evaluated here.
Candidate selection	Which supplied option satisfies these criteria?	Integration pattern; not task-evaluated here.
Review prioritization	Which items deserve a human/agent review first?	Proposed assistive use; not a release gate.

Table 2: Applications and a selection framework

Can the outcome be expressed as a small stable set? Can a reviewer label representative examples consistently? Does the decision occur often enough that the additional dependency and review cost could pay off? If not, keep the logic in the main agent or in ordinary code.

A parser should identify a connection-refused error exactly when its format is known. A semantic classifier is more plausible when the same symptom appears in varied natural-language reports. Even then, a category is a lead for investigation, not a verified root cause.

Do not use the service as the sole approval gate for destructive actions or as a replacement for tests. Always leave a path for incomplete evidence.

4 LAYA MCP INTEGRATION

Laya MCP Server was built to expose bounded judgments consistently to multiple coding clients, without embedding project-specific rules into each client. The integration keeps one service interface, explicit metadata and readable operational guidance. [2, 5, 6]

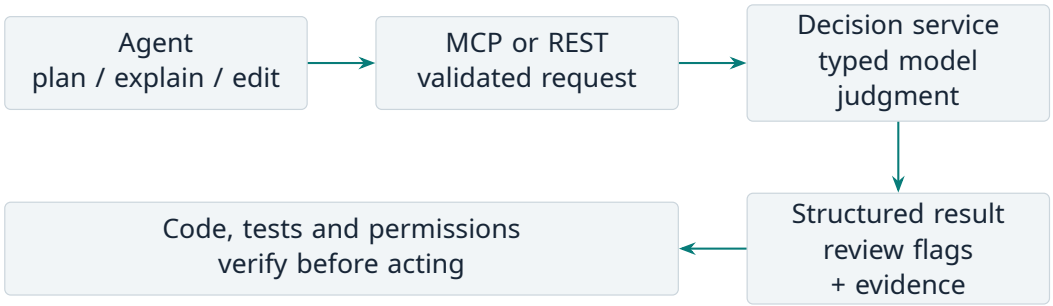


Figure 1: Integration architecture. The service predicts; the agent and application decide how to proceed.

Design need	Project response
Portable access	MCP tools and a normalized REST API share the same decision contract.
Low-friction adoption	One setup entry registers detected user-global clients and a shared skill.
Discoverability	Guide tool/resource, HTTP documentation, Swagger and copyable settings.
Debuggability	Caller/task labels, trace IDs, bounded logs and explicit backend errors.

Table 3: Design requirements and implementation choices

The service predicts values; it does not execute them. Global installation makes tools available across the installing user's projects, not across all machines or users. The operator remains responsible for endpoint access and data handling.

5 DECISION CONTRACTS

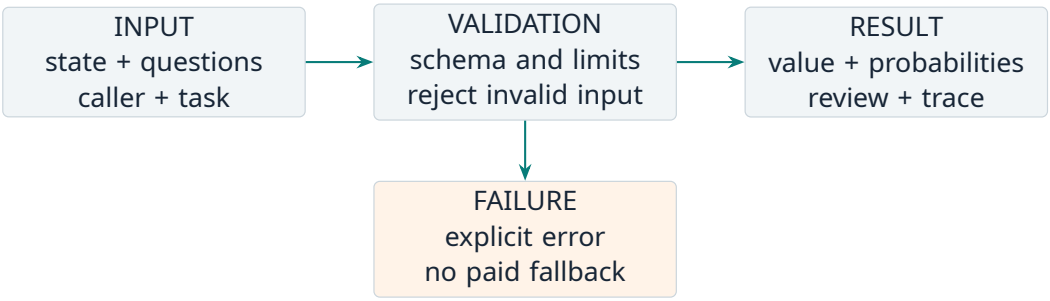


Figure 2: Validation separates a well-formed request, a usable result and an explicit failure.

Type	Intended use	Current limits
choice	Pick a known label	2-10 descriptions; include a review option when needed.
noul	Estimate a yes/no proposition	Always review-required in this release.
score	Rate ordered descriptions	2-5 levels; experimental, always review-required.

Table 4: Supported question types and their limits

laya_decide and POST /v1/decide share a normalized schema. Send short state, one to three questions, caller and task. Each question has a unique ID, type and instructions; criteria depend on the type.

Output	How to read it
value / probabilities	A prediction and its distribution, not a verified fact.
needs_review / warnings	Policy hints and known limitations. false does not authorize action.
usage / duration_ms / trace_id	Diagnostics for the request, not an evaluation of task success.

Table 5: Fields in the normalized response

Malformed or truncated normalized output is rejected. The current question budget is 512 tokens; keep evidence and criteria concise. Confidence is a distribution statistic, not the probability that the answer is correct. [4]

For a choice question with K labels, let p_i be the reported probability of label i, and let $p_{(1)}$ and $p_{(2)}$ be the two largest values. The predicted label and its margin are

$$\hat{y} = \arg\max_{1 \le i \le K} p_i, \qquad m = p_{(1)} - p_{(2)}. \tag{1}$$

The current implementation flags a choice for review when

$$r = \mathbf{1}\left[p_{(1)} < 0.85 \ \vee \ m < 0.25 \ \vee \ \hat{y} \in \mathcal{U}\right], \tag{2}$$

where $\mathcal{U}$ contains the case-insensitive labels `unknown`, `other`, `review` and `uncertain`. This is an implementation heuristic, not a calibrated acceptance rule. Noul and score results are always marked for review in this release. A false review flag does not grant permission to act.

For the connection example, $m = 0.9976 - 0.0014 = 0.9962$. Neither numerical condition in Equation (2) is met, and the selected label is outside $\mathcal{U}$, so the service reports `needs_review=false`. The label still requires an operational check before a change is made.

6 CODING WORKFLOW

Scenario: investigate an application error and make the smallest change that fixes it without breaking an existing valid path. This is a recommended working method, not an automatic feature of the service.

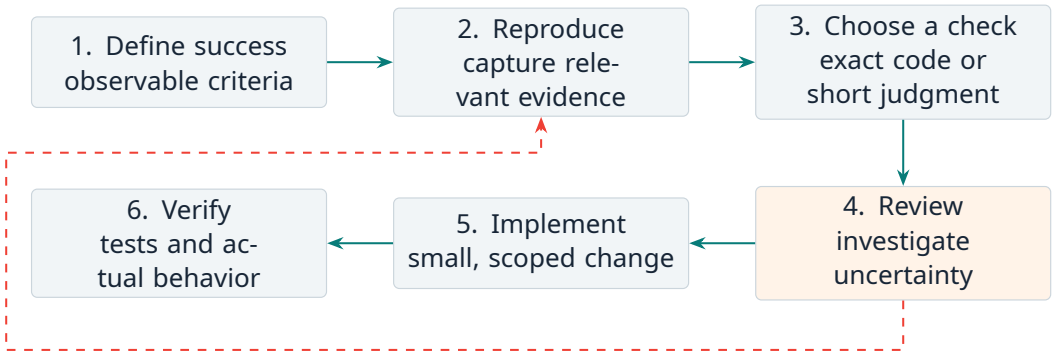


Figure 3: Proposed coding workflow. Uncertain cases return to evidence collection.

State the failing input, expected behavior and regression boundary. Ask the agent to inspect the responsible code before editing.

Capture the exact command, output and relevant log. If a deterministic test already identifies the condition, use that result directly.

Use Laya to classify the failure supported by the log. Investigate its root cause through separate checks.

When evidence is missing or needs_review is true, gather more information. Even a plausible label only prioritizes the next check.

Make a scoped change, rerun the failing case and the unaffected path, then report commands, outcomes and remaining uncertainty.

7 USAGE EXAMPLE

```
./setup.sh
```

Run from the project checkout as the normal user. Follow the project README for prerequisites and deployment details. Start a fresh agent session after global registration.

Entry point	Default path
Test console / Swagger	http://127.0.0.1:30765/ and /docs
MCP / normalized REST	/mcp and POST /v1/decide
Agent-readable guide	/llms.txt, /guide.md or MCP laya_guide

Table 6: Service entry points

```
{
  "state": "ConnectionError: database connection refused at localhost:5432.",
  "questions": [{
    "id": "category", "type": "choice",
    "instructions":
      "Classify only the failure directly shown by this log, not an unproven root cause.",
    "criteria": {
      "connectivity": "Cannot connect to a service",
      "application": "Application logic or assertion failure",
      "unknown": "Insufficient evidence"
    }
  }],
  "caller": "codex", "task": "debug-db-connection"
}
```

```
curl -fsS http://127.0.0.1:30765/v1/decide \
  -H 'Content-Type: application/json' \
  --data-binary @request.json
```

Save the JSON as request.json. The same fields can be sent through laya_decide or Swagger. caller/task labels are for tracing, not identity verification.

The integration can use the review flag to choose the next investigation step without executing a model-selected action:

```
decision = result["decisions"][0]
if decision["needs_review"]:
    next_step = "collect_more_evidence"
elif decision["value"] == "connectivity":
    next_step = "check_service_endpoint"
else:
    next_step = "inspect_application_log"
print(next_step)
```

This snippet illustrates control flow only. It does not verify service availability or modify the application. In the observed connection case, the next step is to inspect the endpoint from the

environment where the error occurred.

8 RESULTS AND ERRORS

A controlled smoke observation on 04 October 2026 used three author-selected examples: one warm-up and five measured calls per example. Expected labels were chosen first. Repetitions do not create fifteen independent test cases.

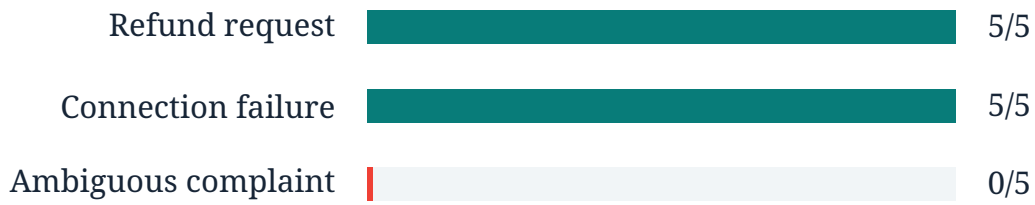


Figure 4: Expected-label matches across five repetitions of each example; there are only three distinct examples.

Example	Expected / observed	Review
Explicit refund request	billing / billing	false
Connection refused log	connectivity / connectivity	false
”Có lỗi rồi, giúp tôi xử lý.”	unknown / application	true

Table 7: Observed labels and review flags

```
value: connectivity
probabilities: connectivity=0.9976, application=0.0014, unknown=0.001
margin: 0.9962
needs_review: false
warnings: [uncalibrated_for_your_task]
advisory: true
```

The connection label is consistent with the supplied log. It does not prove the database is down. Check the listener, endpoint configuration and connectivity from the actual execution environment before deciding what to change.

The ambiguous example produced P(application)=0.6364 and P(unknown)=0.0988. A review flag helped here; this does not prove that every wrong answer will be flagged.

9 EVALUATION

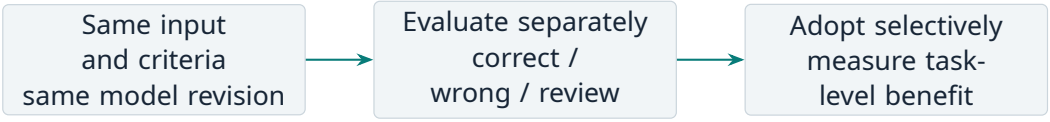


Figure 5: Evaluation procedure for adopting a decision service.

```
Read laya_guide once when this service is relevant.
Prefer one short choice question with clear labels.
Pass the real caller and task purpose.
Treat every result as advisory; inspect uncertainty.
Use code and tests for exact checks.
Do not execute an action merely because the model approves it.
No unbounded retries or automatic paid fallback.
```

Compare complete tasks with and without the extra decision call. Count wrong decisions, review work, repeated calls and recovery time. No controlled end-to-end productivity comparison is claimed in this report.

Build a task-specific labeled set, including ambiguous and out-of-scope inputs. Separate evaluation examples from the examples used to tune questions. Keep the criteria and model revision fixed when comparing variants.

The service provides compact request logs and optional bounded JSON debugging through the configured deployment. Use trace IDs to join related requests; do not add the timing of nested calls. Avoid placing credentials or private content in test data.

A dependency-compatible interface does not imply equivalent model quality. Laya's multilingual card documents calibration and scoring limitations. [3]

Review behavior should be measured separately from prediction correctness. Let $a_i = 1$ when case i proceeds without additional review and $a_i = 0$ otherwise. Coverage and the error rate among accepted cases are

$$C = \frac{\sum_{i=1}^{N} a_i}{N}, \qquad R = \frac{\sum_{i=1}^{N} a_i \, \mathbf{1}[\hat{y}_i \neq y_i]}{\sum_{i=1}^{N} a_i}. \tag{3}$$

Here R is defined only when at least one case is accepted. Reporting both quantities exposes the trade-off between accepting more cases and accepting incorrect predictions, a concern studied in selective prediction [9].

If the three distinct examples in this report were accepted whenever `needs_review=false`, coverage would be $C = 2/3$ and the observed accepted-case error would be $R = 0/2$. The remaining case would go to review. These counts describe only those three examples; they cannot establish a deployment threshold or a general accuracy estimate.

10 LIMITATIONS

This narrative survey supports integration decisions but does not establish a general benchmark. The observational examples are too few for an accuracy estimate. Project-specific report content and design are attributed to AIAI Laboratory and htrnguyen; referenced models, code and publications retain their original authorship.

REFERENCES

[1] Avi Chawla (2026). Jev, Clearly Explained. https://blog.dailydoseofds.com/p/jev-clearly-explained.

[2] Convai Innovations. Laya: source and documentation. https://github.com/NandhaKishorM/laya.

[3] Convai Innovations. Laya multilingual model card. https://huggingface.co/convaiinnovations/laya-multilingual.

[4] TypeSafe AI. Confidence reference. https://docs.typesafe.ai/confidence.

[5] Model Context Protocol contributors. Protocol specification. https://modelcontextprotocol.io/specification/latest.

[6] AIAI Laboratory / htrnguyen. Laya MCP Server, v0.1.0. https://github.com/AIAI-Laboratory/laya-mcp-server.

[7] Anthropic (2024). Building effective agents. https://www.anthropic.com/engineering/building-effective-agents.

[8] Ong et al. (2024). RouteLLM: Learning to Route LLMs with Preference Data. https://arxiv.org/abs/2406.18665.

[9] Geifman & El-Yaniv (2019). SelectiveNet. ICML 2019. https://arxiv.org/abs/1901.09192.

[10] Anthropic (2025). Effective context engineering for AI agents. https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents.

[11] Anthropic (2026). Demystifying evals for AI agents. https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents.